Programming Fundamentals Lab Assignment

Verified Coding Solutions compiled according to strict Verification Guidelines

Question 41: Hello World Program

1. PROBLEM STATEMENT

Write a comprehensive program in C++ to display the standard text string "Hello World" on the console terminal screen, followed by a newline execution.

2. ALGORITHM / STEPS

1. Start the program execution.
2. Include the standard Input/Output library header `<iostream>`.
3. Utilize the standard namespace domain (`std`).
4. Define the `main()` driver entry point function.
5. Stream the character string literal `"Hello World"` to the console output object `cout`.
6. Append the stream manipulator `endl` to force a newline and flush the screen buffer.
7. Return status code 0 indicating clean operational success and exit.

3. DRY RUN WITH SAMPLE INPUT

Input: None required.

Execution Tracing: The program encounters the `cout` statement. It extracts bytes from the string constant literal storage, sequentially sends them directly into the terminal screen buffer, and then applies a line break sequence.

4. PROGRAM CODE (C++)

```
#include <iostream>
using namespace std;

int main() {
    // Printing the message to the console
    cout << "Hello World" << endl;
    return 0;
}
```

5. OUTPUT VERIFICATION LOG

[Console Output State Log]
Hello World

Process exited with return value 0

6. EXPLANATION OF THE LOGIC USED

The program leverages the standard runtime framework of C++. The statement `#include <iostream>` enables access to streaming utilities. The expression `cout <<` sends data out sequentially to the terminal screen interface. Returning an integer value of `0` conveys an error-free termination context back to the host operating system platform.

Question 42: Sum of Two Numbers

1. PROBLEM STATEMENT

Write a program to securely accept two unique integer numerical inputs from a user via the keyboard terminal interface, compute their cumulative sum arithmetic, and then output the final calculation on screen.

2. ALGORITHM / STEPS

1. Start.
2. Declare integer variables named `num1`, `num2`, and `sum`.
3. Display a prompt text string demanding input records from the operating user.
4. Use the standard input stream `cin` to catch and assign data into `num1` and `num2` sequentially.
5. Compute the addition operation: `sum = num1 + num2`.
6. Print the evaluated value stored inside the `sum` location with clear contextual messaging text.
7. Terminate.

3. DRY RUN WITH SAMPLE INPUT

Sample Input: `15 25`
Tracking Matrix:

- `num1` is initialized as 15; `num2` is initialized as 25.
- Calculation step evaluated: $sum = 15 + 25 = 40$.
- Console out reads value: `40`.

4. PROGRAM CODE (C++)

```
#include <iostream>
using namespace std;

int main() {
    int num1, num2, sum;

    cout << "Enter two integers: ";
    cin >> num1 >> num2;

    // Computing arithmetic addition
    sum = num1 + num2;

    cout << "The sum is: " << sum << endl;
    return 0;
}
```

5. OUTPUT VERIFICATION LOG

[Console Output State Log]
Enter two integers: 15 25
The sum is: 40

6. EXPLANATION OF THE LOGIC USED

Variables act as isolated safe zones in system RAM. `cin >>` acts defensively, reading terminal keyboard inputs and matching them safely against integer definitions. The + operator represents binary additions on raw memory addresses, feeding a terminal computational result downstream into `cout`.

Question 43: Area of a Rectangle

1. PROBLEM STATEMENT

Write a program that prompts a user to supply both length and width floating-point parameters belonging to a geometric rectangle, calculates the overall two-dimensional surface area, and outputs the result.

2. ALGORITHM / STEPS

1. Start.
2. Declare high-precision decimal storage variables `length`, `width`, and `area` utilizing the `double` keyword structure.
3. Output a literal clean console request asking for dimension metrics.
4. Execute stream reading steps via `cin` for both dimensional values.
5. Apply the geometric algorithm formula: $area = length \times width$.
6. Deliver the evaluated `area` variable back to the screen via `cout`.
7. End program execution parameters.

3. DRY RUN WITH SAMPLE INPUT

Input Parameters: Length = `5.5`, Width = `4.0`
Evaluation trace: Variables `length` holds 5.5, `width` holds 4.0. The math unit multiplies 5.5 by 4.0 yielding a real result value equal to `22.0`. `area` prints as 22.

4. PROGRAM CODE (C++)

```
#include <iostream>
using namespace std;

int main() {
    double length, width, area;

    cout << "Enter length and width of rectangle: ";
    cin >> length >> width;

    // Calculating the mathematical area
    area = length * width;

    cout << "Area of rectangle = " << area << endl;
    return 0;
}
```

5. OUTPUT VERIFICATION LOG

[Console Output State Log]
Enter length and width of rectangle: 5.5 4.0
Area of rectangle = 22

6. EXPLANATION OF THE LOGIC USED

The data parameters are set as `double` types rather than raw integers to allow handling real-world fractional precision numbers without risking rounding truncation downfalls. The multiplication symbol * triggers floating-point compute registers to process dimensions safely and reliably.

Question 45: Simple Interest Calculator

1. PROBLEM STATEMENT

Write an evaluation program to compute financial Simple Interest yields based on distinct parameters input by the user: Principal sum, Rate of annual interest, and active Time duration in years.

2. ALGORITHM / STEPS

1. Start.
2. Declare double variables: `principal`, `rate`, `time`, and `simple_interest`.
3. Output user directives requesting financial configuration values.
4. Collect active numeric details into matching destination elements via `cin`.
5. Apply formula logic: ***simple_interest* = *(principal* × *rate* × *time)* / *100.0***.
6. Render the resultant evaluation onto the stream path alongside an explainer string.
7. Terminate program sequence context.

3. DRY RUN WITH SAMPLE INPUT

Input Matrix: Principal = `1000`, Rate = 5, Time = 2

Trace: ***simple_interest* = *(1000* × *5* × *2)* / *100.0* = *10000* / *100.0* = *100.0***. The variable `simple_interest` yields exactly 100.

4. PROGRAM CODE (C++)

```
#include <iostream>
using namespace std;

int main() {
    double principal, rate, time, simple_interest;

    cout << "Enter Principal, Rate of Interest, and Time (years): ";
    cin >> principal >> rate >> time;

    // Evaluating utilizing explicit floating-point division
    simple_interest = (principal * rate * time) / 100.0;

    cout << "Simple Interest = " << simple_interest << endl;
    return 0;
}
```

5. OUTPUT VERIFICATION LOG

[Console Output State Log]
Enter Principal, Rate of Interest, and Time (years): 1000 5 2
Simple Interest = 100

6. EXPLANATION OF THE LOGIC USED

The evaluation utilizes explicit division by a floating constant scalar value (`100.0`) rather than a clean integer literal `100` to prevent logic truncation downfalls in complex boundary conditions. This keeps internal data lines operating inside a high-precision state throughout computation.

Question 47: Swapping Two Numbers

1. PROBLEM STATEMENT

Write a functional memory manipulation block to successfully swap/exchange values held inside two discrete variable compartments using a auxiliary temporary memory address variable.

2. ALGORITHM / STEPS

1. Start program.
2. Declare primary storage targets a and b, alongside an intermediate buffer asset `temp`.
3. Accept values from keyboard into destinations a and b.
4. Back up structural value content inside variable a into temporary holder: `temp = a`.
5. Overwrite initial area a safely using information from source b: `a = b`.
6. Overwrite the values inside target b utilizing initial backup records held within cache: `b = temp`.
7. Output swapped values onto console interface.
8. Terminate context.

3. DRY RUN WITH SAMPLE INPUT

Input Parameters: Initial states: `a = 10`, `b = 20`.
Swap Trace Workflow:

- `temp = a` → `temp` assumes copy value 10.
- `a = b` → Variable a overwrites past value to assume 20.
- `b = temp` → Variable b safely assumes state tracking value 10 from cache storage.

Final Result State: Variable a now contains value 20; variable b contains value 10.

4. PROGRAM CODE (C++)

```
#include <iostream>
using namespace std;

int main() {
    int a, b, temp;

    cout << "Enter value for a and b: ";
    cin >> a >> b;

    cout << "Before Swapping: a = " << a << ", b = " << b << endl;

    // Executing memory container rotation sequence
    temp = a;
    a = b;
    b = temp;

    cout << "After Swapping: a = " << a << ", b = " << b << endl;
    return 0;
}
```

5. OUTPUT VERIFICATION LOG

[Console Output State Log]
Enter value for a and b: 10 20
Before Swapping: a = 10, b = 20
After Swapping: a = 20, b = 10

6. EXPLANATION OF THE LOGIC USED

Direct copy action across operational lines like a = b ruins native data contents because the storage location can hold only a single unique configuration pattern at an instance. Introducing `temp` provides an intermediary holding zone to safely isolate historical configuration values before mutations take place.